강화학습 교육 3일차
humanoid_RL_project

이름: 김도훈
학번:2026406041
학과:로봇학부

<목차>
1. 환경 설명
2. 코드 분석
3. 실험 및 분석

1. 환경 설명

Humanoid-v5 환경은 2족 휴머노이드 로봇을 제어하는 환경이다. ▌

Goal(목표)

휴머노이드 로봇이 넘어지지 않고 앞으로 최대한 빠르게 이동하도록 하는 것

Action Space(행동)

17차원 연속된 값을 가진다. 각 액추에이터에 −0.4 ~ 0.4 사이의 제어 명령을 내려 힌지 관절에 토크를 발생시킨다. 공식 문서에 17개의 값이 표로 잘 정리되어 있다. Humanoid-v5 환경에서 17개의 값이 필요한 이유는 사람의 보행 동작과 로봇의 관절 구조를 연결하여 이해할 수 있다. 사람은 걸을 때 고관절과 무릎뿐만 아니라 허리와 팔도 움직이며 자세와 균형을 유지한다. Humanoid-v5 역시 이러한 움직임을 묘사하기 위해 17개의 액추에이터로 구성되어 있는 것이다.

Observation space(관측)

휴머노이드 로봇이 어떤 상태인지 나타내는 정보이고 총 348개가 있다. 로봇에 제어 명령을 내릴 땐 단순히 명령만 내리는 것이 아니다. 로봇의 위치, 자세, 각 관절의 위치, 속도 정보들을 고려하여 제어 명령을 내린다. 아래는 PPO 알고리즘에서 이러한 정보들을 고려하여 학습하기 위해 만들어진 값들이다.

Qpos: 로봇 몸통의 위치와 방향, 각 관절의 각도 등을 나타내는 정보이다. 로봇이 보행할 때는 현재 관절이 얼마나 회전했는지 몸통이 어느 방향으로 기울어져 있는지 알아야 한다. 이러한 정보를 이용하여 현재 로봇의 자세를 파악하고 다음 행동을 결정한다.

Qvel: 로봇 몸통의 선속도와 각속도 각 관절의 회전 속도를 나타내는 정보이다. 로봇이 걷기 위해서는 현재 자세뿐만 아니라 몸통과 관절이 어떤 방향으로 얼마나 빠르게 움직이고 있는지도 알아야 한다. 이러한 속도 정보를 이용하여 보행 중 움직임을 조절하고 균형을 유지하기 위한 행동을 학습한다.

Cinert: 로봇을 구성하는 각 강체의 질량과 관성에 관한 정보이다. Humanoid-v5에서는 13개 신체 부분에 대해 각각 10개의 값이 사용되어 총 130개의 정보로 구성된다. 로봇이 움

직일 때 각 신체 부분의 질량과 관성은 로봇의 움직임에 영향을 준다. 따라서 이러한 정보는 로봇의 행동을 결정하는 데 활용될 수 있다.

Cvel: 로봇을 구성하는 각 신체 부분의 질량중심을 기준으로 표현한 선속도와 각속도 정보이다. Humanoid-v5에서는 13개 신체 부분에 대해 각각 6개의 값이 사용되어 총 78개의 정보가 있다. 로봇이 보행할 때 몸통과 팔다리 등 각각의 부분이 다른 속도로 움직인다. 이러한 정보는 보행 중 균형을 유지하기 위한 행동을 결정하는 데 도움이 된다.

qfrc_actuator: 로봇의 액추에이터가 각 관절 자유도에 발생시키는 토크에 관한 정보이다. Humanoid-v5에서는 제어 가능한 17개 관절에 대응하는 값이 사용된다. 로봇이 보행할 때는 관절의 위치와 속도뿐 아니라 각 관절에 어떤 힘이 작용하고 있는지도 중요하다. 이러한 정보를 이용하여 현재 보행 상태를 고려한 다음 행동을 학습할 수 있다.

cfrc_ext: 로봇의 각 신체 부분에 작용하는 외부 힘과 토크에 관한 정보이다. Humanoid-v5에서는 13개 신체 부분에 대해 각각 6개의 값이 사용되어 총 78개의 정보로 구성된다. 보행 중에는 발이 지면에 접촉하면서 힘을 받게 된다. 이러한 외부 힘에 관한 정보는 로봇이 지면과 접촉한 상태를 파악하고 균형을 유지하는 데 활용될 수 있다.

Starting State(초기 상태)

초기 상태는 학습을 시작할 때 로봇이 가지는 위치, 자세, 속도를 의미한다. 공식 문서에 나오는 로봇의 초기 상태는 다음과 같다.

$$[0.0, 0.0, 1.4, 1.0, 0.0, \dots 0.0] + \mathcal{U}_{[-reset_noise_scale \times I_{24}, reset_noise_scale \times I_{24}]}$$

[] 안 값들은 거의 휴머노이드 로봇의 기본 값들이다. 이 값에 u라는 랜던함 값을 더한다. 랜덤한 값을 더하는 이유는 로봇이 매번 완전히 똑같은 자세로 시작한다면 특정 시작 자세에서만 잘 움직이는 정책을 학습할 위험이 있다. 반대로 작은 랜던한 값(무작위 변화)를 추가하면 로봇은 조금씩 다른 초기 상태에서도 균형을 잡는 방법을 경험할 수 있다. 이렇게 하면 다양한 초기 상태에 대응하는 정책을 학습하는 데 도움이 되기 때문이다. Seed 설정 또한 학습에 영향을 주기 때문에 실험 시에는 같은 seed로 설정해두어야 한다.

Episode End(에피소드 종료)

강화학습에서는 두 가지의 종료 조건이 있다. Humanoid-v5 환경도 같은 두 가지 종료 조건을 가진다. 첫 번째 terminated 종료이다. 이 종료 조건은 로봇이 비정상 상태에 도달한 경우 에피소드가 종료되는 조건이다. 공식 문서에서는 z(허리 높이) 1.0 이상 2.0 이하 이 범위를 벗어나면 terminated 종료로 판단한다. 로봇의 기

본 허리 높이는 Z값 1.4이다. 위의 범위는 로봇이 넘어지거나 비정상적으로 높이 올라간 경우를 뜻한다. 그래서 로봇의 안정적인 보행을 학습하기 위해서는 terminated 종료 조건이 있어야 한다. 두 번째 truncated 종료이다. 이 종료 조건은 정해진 시간 제한에 도달한 경우 종료되는 조건이다. 공식 문서에서는 1000 step 도달이 truncated 종료의 조건으로 하고 있다.

Rewards(보상)

휴머노이드 로봇은 과제로 수행했던 frozenlake와 다르게 목적지에 도착하는 것보다는 매 step 마다 어떤 행동을 하는지가 중요하기 때문에 보상은 에피소드가 종료 시 부여되는 것이 아닌 매 step 종료 시 부여된다. 보상 함수는 다음과 같다.

$$reward = healthy_reward + forward_reward - ctrl_cost - contact_cost$$

healthy_reward (건강 상태 보상): 공식 문서에서는 휴머노이드가 살아있는 한 기본 값 5를 받는다고 설명 되어 있다. 이 건강 상태 보상은 위 종료 조건 설명에 나오는 terminated 종료 조건을 만족하지 않으면 기본값 5를 받는다.

Forward Reward(전진 보상): 휴머노이드 로봇이 앞으로 이동한 속도에 따라 부여되는 보상이다. 함수는 다음과 같다.

$$w_{forward} \times \frac{dx}{dt}$$

위 함수처럼 기본 값(W) 1.25에 전진한 속도를 곱한 만큼의 양의 보상을 받는다. 빠른 속도로 앞으로 이동했다면 큰 보상을 받는 것이다.

Control Cost(제어 비용): 휴머노이드 로봇 관절 17개에 입력되는 큰 제어 입력을 억제하는 보상이다. 제어 비용이 필요한 이유를 예를 들어 설명하면 로봇은 학습을 하면서 큰 전진 보상을 받기 위해 관절 17개에 큰 제어 입력을 받는다. 하지만 오히려 큰 제어 입력은 로봇을 불안정하고 전복하게 만들기 때문에 이러한 상황을 해결하고 큰 제어 입력을 억제하여 로봇이 안정적이고 빠르게 걷도록 학습시키는 것이다. (w 기본값 0.1)

$$w_{control} \times \|action\|_2^2$$

Contact Cost(접촉 비용): 휴머노이드 로봇이 외부와 접촉했을 때 발생하는 힘과 토크를 억제하는 보상이다. 접촉 비용이 필요한 이유를 예를 들어 설명하면 로봇은 학습을 하면서 큰 전진 보상을 받기 위해 큰 제어 입력을 받는데 이때 큰 제어 입

력에 의해 로봇과 지면이 접촉했을 때 발생하는 외력이 커질 수 있다. 이것은 과격한 보행을 만들고 로봇을 불안정하게 하여 로봇이 전복되게 하는 원인이 된다. 이러한 상황을 해결하기 위해 외부와 접촉했을 때 발생하는 힘과 토크를 억제하여 로봇이 자연스럽고 부드럽게 보행할 수 있도록 하는 것이다.

2. 코드 분석

이번 humanoid 강화학습 프로젝트에서는 PPO 알고리즘을 직접 구현한 것 아닌 Stable-Baselines3 라이브러리(버전 2.9.2)의 구현된 PPO 알고리즘을 이용해 학습을 시켰다. 그리고 관련된 라이브러리에 대한 학습은 구글링 자료[1] 와 생성형 AI를 통해 학습하였다.

사용한 라이브러리 깃허브 저장소 링크:
https://github.com/DLR-RM/stable-baselines3/tree/master/stable_baselines3
ppo.py:
https://github.com/DLR-RM/stable-baselines3/blob/master/stable_baselines3/ppo/
On_policy_algorithm.py
https://github.com/DLR-RM/stable-baselines3/blob/master/stable_baselines3/common/on_policy_algorithm.py
Buffers.py:
https://github.com/DLR-RM/stable-baselines3/blob/master/stable_baselines3/common/buffers.py

model.learn(total_timesteps = 204_800)

위 코드는 ppo_train,py의 Model 학습 코드이다. 원하는 timestep 만큼 반복하여 model을 학습 시킬 수 있다. 위 코드를 사용해 원하는 timestep를 입력해주면 On_*policy*_algorithm.py의 learn() 함수에서는 입력한 timestep만큼 경험을 수집하고 PPO을 학습을 반복하도록 한다.

```
while self.num_timesteps & total_timesteps:
            continue_training = self.collect_rollouts(self.env, callback, self.rollout_buffer,
n_rollout_steps=self.n_steps)
            if not continue_training:
                break
            iteration += 1
            self._update_current_progress_remaining(self.num_timesteps, total_timesteps)
```

---

1) 남하욱, stable baselines 3 사용법 기초, https://namhauk.tistory.com/35

```
            # Display training infos
            if log_interval is not None and iteration % log_interval == 0:
                assert self.ep_info_buffer is not None
                self.dump_logs(iteration)

            self.train()
```

이 코드는 learn() 함수 내에 있는 경험을 수집하는 while 문이다. 이 while문은 우리가 입력해준 timestep만큼 반복한다.

```
continue_training = self.collect_rollouts(self.env, callback, self.rollout_buffer,
n_rollout_steps=self.n_steps)
```

학습 중에 위와 같은 코드를 사용해 경험을 수집한다. 함수의 입력되는 주요 매개변수들은 self.env = humanoid-v5 환경, rollout*buffer = 수집한 경험을 저장할 버퍼, n_rollout*_steps = 환경마다 수집할 스텝 수, 기본 2048개 이다. 그 다음 수집한 경험을 저장하고 얼마나 좋은 행동이었는지 PPO가 판단할 수 있도록 return과 advantage를 계산하다.

```
self.observations[self.pos] = np.array(obs)
        self.actions[self.pos] = np.array(action)
        self.rewards[self.pos] = np.array(reward)
        self.episode_starts[self.pos] = np.array(episode_start)
        self.values[self.pos] = value.clone().cpu().numpy().flatten()
        self.log_probs[self.pos] = log_prob.clone().cpu().numpy()
        self.pos += 1
        if self.pos == self.buffer_size:
            self.full = True
```

위 buffers.py 코드에 있는 add() 함수를 이용해 수집한 경험을 버퍼에 저장하는 코드이다. 시뮬레이션에서 로봇이 한 번 행동할때마다 경험을 저장한다.
buffers.py의 compute_returns_and_advantage() 함수를 사용해 return과 advantage를 계산한다.

```
def compute_returns_and_advantage(self, last_values: th.Tensor, dones: np.ndarray) -> None:
        last_values = last_values.clone().cpu().numpy().flatten()  # type: ignore[assignment]
        last_gae_lam = 0
        for step in reversed(range(self.buffer_size)):
            if step == self.buffer_size - 1:
                next_non_terminal = 1.0 - dones.astype(np.float32)
                next_values = last_values
            else:
```

```
                next_non_terminal = 1.0 - self.episode_starts[step + 1]
                next_values = self.values[step + 1]
            delta = self.rewards[step] + self.gamma * next_values * next_non_terminal - self.values[step]
            last_gae_lam = delta + self.gamma * self.gae_lambda * next_non_terminal * last_gae_lam
            self.advantages[step] = last_gae_lam
        self.return=self.advantages + self.values
```

Compute_returns_and_advantage() 함수 코드이다. 이 함수는 저장된 마지막 스텝부터 첫 번째 스텝까지 역순으로 반복한다. 계산된 값들을 PPO 학습 함수에 전달할 수 있도록 RolloutBuffer.get()를 사용한다.

3. 실험 및 분석

실험 주제

PPO 알고리즘을 사용해 학습한 모델의 성능에 학습율은 영향을 줄까?

실험 조건

	모델 A	모델 B
환경	humanoid-v5	humanoid-v5
알고리즘	PPO	PPO
학습 step	204800	1024000
평가 횟수	20	20

먼저 PPO 모델을 204800스텝 동안 학습한 후 모델을 저장하였다. 이후 저장된 모델을 불러와 추가로 819200스텝을 학습하여 누적 1024000스텝에 도달하도록 하였다.

사전 가설 설정

PPO 알고리즘을 사용해 학습한 모델의 성능에 학습율은 영향을 준다.

고정 조건

고정 항목	설정
환경	humanoid-v5
알고리즘	PPO
Batch size	64
할인율	0.99
학습 장치	CPU

시작 조건

ppo_train.py로 정해진 step 만큼 모델이 학습되도록 했고 ppo_test.py로 같은 환경에서 모델만 변경하여 테스트를 진행했다.

측정 결과

모델 A
평균 Episode Reward: 323.42118595
Episode Reward 표준편차: 21.36745876599082
평균 Episode Length: 64.0
최소 Episode Length: 58
최대 Episode Length: 72

모델B
평균 Episode Reward: 421.3539184
Episode Reward 표준편차: 107.99044534385288
평균 Episode Length: 78.85
최소 Episode Length: 56
최대 Episode Length: 133

학습 스텝을 증가시킨 결과 평균 Episode Reward 값은 약 98 증가하였고 평균 Episode Length는 약14.85 증가하였다. 반면 Reward 표준편차는 21.36745876599082에서 107.99044534385288로 증가하였다. Episode Length의 최솟값은 두 모델 모두 비슷했으나 최댓값은 72 스텝에서 133 스텝으로 증가하였다. 실제 MuJoCo 시뮬레이션에서는 학습된 로봇이 앞으로 조금 이동하다가 넘어지는 경우와 제자리에서 바로 넘어지는 경우가 모두 관찰되었다. 따라서 평균 보상과 Episode 유지 시간이 증가한 것은 확인하였지만 안정적으로 걷는 동작이 완성되지는 않았다.

결과 해석
학습 스텝 수가 증가하면서 모델 A보다 모델 B의 평균 Episode Reward와 Episode Length가 동시에 증가한 것은 PPO가 추가 학습을 통해 더 오래 Episode를 유지하는 행동을 학습했다는 것을 보여준다. 그러나 평균 보상 증가가 안정적인 보행 능력 향상을 의미하는 것은 아니다. Humanoid-v5는 자세 유지, 전진 속도, 제어 입력에 따른 보상도 함께 고려한다. 따라서 로봇이 실제로 안정적인 걸음을 수행하지 못하더라도 자세를 조금 더 오래 유지하는 것만으로 누적 보상이 증가할 수 있다. 실제 시뮬레이션에서도 일부 전진 동작이 나타났지만 넘어지는 현상이 계속해서 발생했다. 이는 누적 보상과 실제 보행 안정성 사이에 차이가 있음을 보여준다.

결론

학습율의 증가가 평균 누적 보상과 자세 유지 시간의 개선은 가능하지만 이러한 개선만으로는 안정적이 보행 정책을 학습했다고 판단할 수 없다.